公開鍵ベース ZK とハッシュ関数ベース透明 ZK の応用比較

Ligero, zk-STARK, Aurora, Virgo を中心に

2026 年 3 月 25 日

概要

本稿は、ゼロ知識証明の応用・ユースケースを、**公開鍵ベースの ZK** と**ハッシュ関数ベースの透明 ZK** に大別しつつ、後者の代表として Ligero, zk-STARK, Aurora, Virgo を比較・整理する独立ノートである。狙いは、単なる理論比較ではなく、「どの用途で、なぜその方式が選ばれやすいのか」を設計判断の観点からまとめることにある。

本稿では「公開鍵ベース」を、楕円曲線群・ペアリング・KZG/IPA/Pedersen などの公開鍵暗号的な代数構造に本質的に依存する ZK という広い意味で使う。その中には、Groth16, PLONK, Marlin, Halo/Halo2, Bulletproofs などが含まれる。一方「ハッシュ関数ベース」は、衝突耐性ハッシュ、Merkle 木、符号理論、低次数検査、**IOP** を主な土台にする系を指す。この二分法と「trusted setup の有無」は別軸である。たとえば Groth16 は relation-specific setup を要し、PLONK/Marlin は universal / updatable SRS を使い、Halo は trusted setup なしの公開鍵ベースである。これに対し Ligero, STARK, Aurora, Virgo は transparent である [1, 2, 3, 4, 7, 8, 11, 13, 14]。

重要なのは、実務上の選択が「理論上どれが強いか」だけでは決まらない点である。オンチェーン検証コスト、proof size、prover time と memory、transparent であることの監査容易性、量子計算耐性の見通し、分散証明のしやすさ、回路モデル（**R1CS**, **AIR**, layered circuits, Boolean circuits）の相性、ツールチェーンの成熟度などが絡み合う。したがって本稿の推薦は、原論文の性質、公開されている実装・プロダクト、およびそれらから導かれる設計上の含意を総合した**実務的な推奨**である。特にユースケース節では、文献上の厳密な主張と、そこからの自然な設計推論を区別して述べる [11, 16, 17, 19, 24, 15]。

目次

1 本稿の立場と用語 4
1.1 「公開鍵ベース」と「ハッシュ関数ベース」の区別 4
1.2 本稿での比較対象 4

2 比較軸 4
2.1 比較軸の一覧 4

2.2 なぜこの軸が必要か 5

3 公開鍵ベース ZK の代表的ファミリーとユースケース 5
3.1 Groth16 5
3.2 PLONK と Plonkish 系 6
3.3 Marlin 6
3.4 Halo / Halo2 7
3.5 Bulletproofs 7

4 ハッシュ関数ベース透明 ZK の共通する特徴 8
4.1 透明性 8
4.2 post-quantum plausibility 8
4.3 proof size と verifier cost の傾向 8

5 zk-STARK の応用とユースケース 8
5.1 なぜ STARK は「大きな計算」に向くのか 8
5.2 代表ユースケース 1: zkVM / verifiable CPU execution 9
5.3 代表ユースケース 2: rollup / shared proving / batching 9
5.4 代表ユースケース 3: 透明性や長期監査が重要な計算証明 9
5.5 弱点と実務上の補完策 10

6 Ligero の応用とユースケース 10
6.1 Ligero の位置づけ 10
6.2 どの用途に向くか 10
6.3 Boolean circuits と BooLigero 10
6.4 Ligero の弱点 11
6.5 実務的なまとめ 11

7 Aurora の応用とユースケース 11
7.1 Aurora の位置づけ 11
7.2 どの用途に向くか 11
7.3 実務的な向き先 12
7.4 弱点と歴史的位置づけ 12

8 Virgo の応用とユースケース 12
8.1 Virgo の位置づけ 12
8.2 どの用途に向くか 12
8.3 代表ユースケース 1: zkML 13
8.4 代表ユースケース 2: cross-chain bridge 13

8.5 代表ユースケース 3: distributed proving . . . 13
8.6 Virgo の弱点と注意点 . . . 13

9 **公開鍵ベース ZK とハッシュ関数ベース透明 ZK の包括比較** 14
9.1 大きな傾向 . . . 14
9.2 比較表：公開鍵ベース vs. ハッシュベース . . . 14
9.3 設計判断としてどう読むべきか . . . 15

10 **ハッシュ関数ベース四方式の横比較** 15
10.1 高レベル比較 . . . 15
10.2 実務的な含意 . . . 16

11 **典型ユースケース別の選び方** 16
11.1 **EVM** / L1 上で verifier gas を最小化したい . . . 16
11.2 巨大なプログラム実行を証明したい . . . 16
11.3 setup 不要の透明証明が欲しいが、対象は中規模回路である . . . 16
11.4 Merkle trees, batch signatures, zkML, structured pipelines . . . 17
11.5 量子耐性や長期監査を重視する . . . 17
11.6 完全に one-size-fits-all を求めない . . . 17

12 **ハイブリッド構成が重要である理由** 17
12.1 RISC Zero: STARK の上に Groth16 . . . 17
12.2 zkBridge: deVirgo の上に Groth16 . . . 18
12.3 理論的含意 . . . 18

13 **補足的な考察** 18
13.1 透明であることの非技術的価値 . . . 18
13.2 「透明系 = 何でも **PQ**」ではない . . . 18
13.3 Virgo は原版より系譜で捉える方が良い . . . 19
13.4 Aurora と Ligero の現在位置 . . . 19

14 **実務的な結論** 19

15 **結論** 20

1　本稿の立場と用語

1.1　「公開鍵ベース」と「ハッシュ関数ベース」の区別

まず強調しておきたいのは、**公開鍵ベース**かどうかと **trusted setup があるか**どうかは別軸である、という点である。Groth16 は relation-specific trusted setup を必要とする代表例であり、PLONK や Marlin は universal / updatable SRS を用いる代表例である。一方 Halo は trusted setup なしの recursive proof composition を達成した。つまり「公開鍵ベース = 必ず trusted setup あり」ではない [1, 2, 3, 4]。

同様に、**ハッシュ関数ベース**と呼ばれる透明系も、実装では他のサブプロトコルを組み込むことがある。たとえば RISC Zero は AIR / FRI ベースの **STARK** 系でありつつ、memory verification に PLONK 風の grand product accumulator を、range-check に PLOOKUP 系の考え方を用いると説明している。したがって今日の設計空間では、「公開鍵ベース vs. ハッシュベース」は排他的な陣営というより、**主たる証明骨格が何に依存するか** を示すラベルだと理解するのが正確である [19, 20]。

1.2　本稿での比較対象

本稿での中心は次の二群である。

- **公開鍵ベースの代表例**：Groth16, PLONK, Marlin, Halo/Halo2, Bulletproofs
- **ハッシュ関数ベース透明系の代表例**：Ligero, zk-STARK, Aurora, Virgo

ここで公開鍵ベース側は「比較のための参照点」であり、主目的は後者四方式のユースケースの棲み分けを整理することにある。ただし、実務上は両者のハイブリッドが極めて重要であるため、後半では

- STARK を Groth16 でラップする構成
- deVirgo の上に Groth16 を重ねる構成

なども明示的に扱う [17, 18, 24]。

2　比較軸

ユースケースの議論に入る前に、ゼロ知識証明を選ぶときの比較軸を明示しておく。

2.1　比較軸の一覧

1. **setup**：trusted setup が必要か、必要なら relation-specific か universal / updatable か。

2. **暗号仮定**：離散対数・ペアリング・楕円曲線群か、衝突耐性ハッシュ・符号理論・低次数検査か。
3. **proof size**：通信量、保管量、L1 calldata / DA コストへの影響。
4. **verification cost**：特に on-chain verifier の gas、あるいはクライアント側検証時間。
5. **prover time / memory**：巨大計算を捌けるか、単一マシンか分散証明か。
6. **arithmetic model**：**R1CS**, **AIR**, layered circuits, Boolean circuits, RAM / VM traces との相性。
7. **transparency & auditability**：ceremony がないことの監査上の利点。
8. **post-quantum plausibility**：将来の量子計算環境での説明可能性。
9. **recursion / aggregation / batching**：証明圧縮や多数インスタンス集約との相性。
10. **tooling maturity**：コンパイラ、verifier 生成、実運用事例、開発者向け資料の豊富さ。

2.2 なぜこの軸が必要か

同じ「ZK を使いたい」という要望でも、

- **EVM** 上で安く検証したいのか
- 巨大なオフチェーン計算を transparent に証明したいのか
- 量子計算耐性の説明責任が重要なのか
- データ並列・分散証明を使いたいのか

で最適解は大きく変わる。

たとえば **EVM** 向けツールの代表例である `snarkjs-generate-solidity` は Groth16 / PLONK / FFLONK の verifier 生成をサポートしている。これは、**EVM** での小さな proof と明示的な verifier code 生成が今でも重要な設計要件であることを示している。一方で Starknet の SHARP は、複数 Cairo プログラムを 1 つの **STARK** proof にまとめて Ethereum で検証する。こちらは「巨大 proof を個別に投げる」のではなく *batching / aggregation* **でコストを償却する** 発想である [7, 16]。

3 公開鍵ベース ZK の代表的ファミリーとユースケース

3.1 Groth16

Groth16 は pairing-based zkSNARK の代表例であり、proof が 3 つの group elements からなる極めてコンパクトな構成として知られる。この性質は、

- 送信コストが小さい
- verifier が非常に軽い
- on-chain verification に向く

という実務的な利点に直結する。代償として relation-specific trusted setup が必要である [1]。

■ユースケース　Groth16 は特に **L1 / EVM での検証コストが最優先** な場面に向く。実際、`snarkjs-generate-solidity` が Groth16 verifier を出力できること、RISC Zero が最終的な tiny receipt として Groth16Receipt を生成することは、「大きい計算の内部証明は別方式で行っても、最終出口は Groth16 で包む」設計が現実的に重要であることを示している [7, 17, 18]。

■向かない場面　一方で、回路ごとに ceremony を回したくない研究用途、長期アーカイブで setup の正当性を何年も説明し続けたくない用途、あるいは量子耐性を強く意識する場面には向きにくい。このあたりで透明系との役割分担が生じる。

3.2　PLONK と Plonkish 系

PLONK は universal SNARK であり、fully succinct verification を持ち、circuit-independent な setup で複数回路を使い回せる点が大きな特徴である [2]。Groth16 の relation-specific setup と比べると、**回路を頻繁に差し替えるアプリケーション**に向く。

■ユースケース　PLONK 系は

- 汎用アプリケーション
- zkRollup のアプリ回路
- **EVM** 連携
- custom gates / lookups を多用する設計

で広く使われる。`snarkjs` 系ツールが PLONK verifier を Solidity として生成できることも、実務上の利用しやすさを後押ししている [2, 7]。

■向かない場面　完全透明性や **PQ** を重視するなら、やはりハッシュ関数ベースの transparent 系が候補になる。また、巨大な逐次実行トレースを素朴に PLONK 化するのは、**AIR** / trace を直接使う **STARK** 系に比べて自然さを欠く場合がある。

3.3　Marlin

Marlin は universal and updatable SRS を持つ preprocessing zkSNARK である。proof は constant-size で、設計思想としては「Groth16 の verifier efficiency と、universal / updatable setup の運用性のバランスをとる」位置づけにある [3]。

■ユースケース　Marlin は

- universal setup を好む研究・実装
- proof size と verifier cost をかなり抑えたい用途
- pairings / polynomial commitments を前提とする一般用途

に向く。

■位置づけ　ただし産業界での言及頻度は、PLONK 系や Halo2 系に比べると見えにくいことが多い。それでも、**「公開鍵ベースだが relation-specific setup は避けたい」**という設計空間の重要な参照点である。

3.4　Halo / Halo2

Halo は trusted setup なしの recursive proof composition を実現した公開鍵ベース ZK の代表例である [4]。Zcash の NU5 ドキュメントは、ネットワークが Halo proving system に移行し、trusted setup を不要にすることを明言している [22]。

■ユースケース　Halo / Halo2 が向くのは、

- recursion が重要
- trusted setup を避けたい
- それでも公開鍵ベースの compactness をある程度維持したい

という場面である。たとえば shielded transactions や recursive rollup proof のような用途では、この方向がかなり魅力的である [4, 22]。

■位置づけ　ここで重要なのは、**公開鍵ベース = 必ず setup あり** ではない、という例として Halo が非常に教育的だという点である。後述の透明系との比較でも、「transparent かどうか」と「公開鍵ベースかどうか」は別軸で整理すべきである。

3.5　Bulletproofs

Bulletproofs は setup 不要で、proof size が witness size に対して対数的であることを特徴とする。Stanford Applied Crypto Group の解説でも、trusted setup 不要である一方、verification は SNARK より重いと整理されている [5, 6]。

■ユースケース　Bulletproofs は、

- confidential transactions
- range proofs
- setup を避けたい金融・プライバシー用途

に非常に相性がよい。特に「proof size をある程度抑えたいが、verification のコストは SNARK より高くても許容できる」という場面で使いやすい [5, 6]。

■向かない場面　大規模な汎用計算や、**EVM** 上での極小 verification cost が必要な用途では、Groth16 / PLONK / Halo2 や、あるいは **STARK** を wrap した構成の方が好まれやすい。

4　ハッシュ関数ベース透明 ZK の共通する特徴

4.1　透明性

Ligero, STARK, Aurora, Virgo の共通点は、いずれも trusted setup を必要としない transparent 系である点にある [8, 11, 13, 14]。この性質は

- setup ceremony に関する監査コストを削減する
- 一点故障点（single point of failure）を避ける
- 長期にわたる説明責任を軽くする

という運用上の利点を持つ。

4.2　post-quantum plausibility

これらの transparent 系は、群上の離散対数やペアリングに本質的に依存しないため、しばしば「plausibly post-quantum secure」と説明される。STARK 論文自体が post-quantum secure computational integrity を掲げており、Ligero 系や transparent 系の survey / 後続研究でも、この方向が重要な差別化要因として扱われる [11, 8, 9, 27]。

ただし、「透明 = 自動的にすべての構成要素が完全に **PQ**」という意味ではない。実装の細部には PRF、Fiat–Shamir、lookup、permutation arguments などが絡み、実システムでは他系統の技術を部分的に借りることもある。したがって、**主たる証明骨格がハッシュ・符号理論ベース**だと理解するのが正確である [19, 20]。

4.3　proof size と verifier cost の傾向

一般にハッシュベース透明系は、pairing-based SNARK より proof が大きくなりやすい。その代わり、setup を不要にし、巨大計算へのスケーリングや透明性を得る。このトレードオフは、Ligero の $\tilde{O}(\sqrt{n})$ 型通信、Aurora の polylog proof だが $O(n)$ verifier、Virgo の succinct だが layered-structure 依存、STARK の大きめの proof と high-throughput batching というかたちで現れる [8, 13, 14, 11]。

5　zk-STARK の応用とユースケース

5.1　なぜ STARK は「大きな計算」に向くのか

STARK の出発点は computational integrity であり、実行トレースと **AIR** 制約を通じて**逐次計算**や**意味のある大規模計算**を直接表現することにある [11, 12]。このため STARK は、四方式の中で最も自然に

- VM execution
- RAM/CPU trace
- rollup execution
- 巨大な逐次的ワークロード

へ向かう。

5.2 代表ユースケース 1: zkVM / verifiable CPU execution

RISC Zero のドキュメントは、RISC-V Circuit, Recursion Circuit, Groth16 Circuit からなる proving stack を説明しており、内部の大きな証明部分が **STARK** 系であることを明示している。また proof-system-sequence-diagram は、execution trace, memory permutation, lookup arguments を含む AIR-based system として記述している [17, 19, 21]。

したがって STARK の第一の代表ユースケースは、**zkVM** である。CPU 実行の正しさ、RISC-V ゲスト実行、ステップ数の多いプログラム実行を証明したいとき、「行が時刻、列が machine state」を表すトレースという STARK の視点は非常に自然である [11, 21]。

5.3 代表ユースケース 2: rollup / shared proving / batching

Starknet の SHARP は、複数 Cairo プログラムを束ねた unified **STARK** proof を生成し、Ethereum 上で検証する仕組みとして説明されている。これは「1 件 1 proof」ではなく、**多数の計算をまとめて一つの大きな *proof* にすることで検証コストを償却する**という STARK 的な使い方の典型例である [16]。

従って STARK の第二の代表ユースケースは、**rollup / batching / aggregated settlement** である。proof size 自体は pairing-based SNARK より大きくても、大量トランザクションをまとめて証明すれば総コストで勝ちやすい。

5.4 代表ユースケース 3: 透明性や長期監査が重要な計算証明

transparent であり、post-quantum の見通しが比較的良いことから、STARK は

- 長期間保全したい監査ログの正当性証明
- 規制・説明責任が重い業務システム
- setup に依存しないこと自体が重要な verifiable computing

にも相性がよい。ここでは「proof がやや大きい」ことよりも、**何を信頼しているのかが単純である**ことの価値が大きい [11]。

5.5 弱点と実務上の補完策

STARK の弱点は、raw proof をそのまま **EVM** に投げると重くなりやすいこと、proof size が pairings 系より大きいこと、実装上の prover resource が依然として大きいことにある。このため実務では二つの補完策がよく使われる。

- **batching**: SHARP のように多数インスタンスをまとめる
- **wrapping**: RISC Zero のように最後を Groth16Receipt に圧縮する

この事実は、STARK が単体で万能というより、**大きな内部計算の証明器として強く、小さな最終レシートは別方式で作る** のが実務的であることを示している [16, 17, 18]。

6 Ligero の応用とユースケース

6.1 Ligero の位置づけ

Ligero は、verification circuit size に対して平方根スケーリングの通信量を持つ transparent proof system であり、collision-resistant hash function のみを仮定して trusted setup や public-key cryptography を不要にすることを前面に出している [8]。その意味で Ligero は、**「透明・ハッシュベース・sublinear proof」の古典的代表**である。

6.2 どの用途に向くか

Ligero のユースケースを実務的に言い換えると、

- 中規模の回路
- Boolean / bit-heavy circuit
- setup 不要を重視する証明
- proof size の極小化より prover の単純さや実装のわかりやすさを重視する場面

に向く。

これは、Ligero が witness table を spot-check する発想であり、巨大な逐次トレース専用というより**「表にしやすい witness を transparent に検査する」**系統だからである [8]。

6.3 Boolean circuits と BooLigero

BooLigero は Ligero を Boolean circuits 向けに改善した後継であり、public-coin, transparent, plausibly post-quantum secure の性質を保ちながら、Boolean 回路に対して proof size を改善する [9]。

このことから、Ligero 系は

- ルール適合性証明
- bit 条件の大量検査
- access policy / compliance / credential predicates
- MPC-in-the-head 風の transparent hash-only proof

といった用途に相性がよいと推測できる。ここは STARK のような VM execution というより、**中規模・条件判定中心の transparent proof** というイメージである。

6.4 Ligero の弱点

Ligero は proof size を極小にする系ではない。Banquet 論文は、Ligero や Aurora が small circuits like AES には proof size の面で不利になりやすいことを論じている。また Ligero の長所は「透明で単純」な点にあるが、極端に小さい on-chain proof を欲しがる用途では、Groth16 / PLONK などの公開鍵ベースや、wrap されたハイブリッド構成に見劣りすることがある [26]。

6.5 実務的なまとめ

したがって Ligero は、**透明・ハッシュベースの中で、proof size 極小化より実装の素朴さと中規模効率を重視する**用途に向く。現代では raw Ligero 単体より、BooLigero や Ligero++ の系譜として理解する方が自然である [9, 10]。

7 Aurora の応用とユースケース

7.1 Aurora の位置づけ

Aurora は transparent で、**R1CS** を直接ターゲットにする zkSNARK として設計された。proof size は $O(\log^2 n)$、prover は $O(n \log n)$、verifier は $O(n)$ とされる [13]。

この構成は重要である。なぜなら Aurora は、**既存の R1CS / arithmetic-circuit の資産を活かしながら、trusted setup を外したい**という需要にまっすぐ応えるからである。

7.2 どの用途に向くか

Aurora が向くのは、

- 既存の **R1CS** パイプラインを透明化したい
- TinyRAM や arithmetic programs を transparent に扱いたい
- verifier がオフチェーンであり、線形コストを受け入れられる
- pairings や structured reference string を避けたい

といった用途である。原論文も TinyRAM の正しい実行証明を重要応用として扱っている [13]。

7.3 実務的な向き先

この性質から、Aurora は

- オフチェーン監査
- 企業内 verifiable computation
- 研究プロトタイプとしての transparent **R1CS** proof
- 既存回路コンパイラの透明 backend

に向くと考えられる。

ここで大事なのは、Aurora が「hash-based 版の Groth16」として最適、という意味ではないことである。proof は短いが verifier は $O(n)$ なので、**最終検証コストを極小化したい EVM 用途**にはそのままでは向きにくい [13]。

7.4 弱点と歴史的位置づけ

Aurora の弱点は verifier 線形性にある。このため Aurora は、今日の実プロダクトで raw 採用されるというより、**transparent R1CS 系の重要な祖先**として理解した方が正確である。Polaris が「transparent **R1CS** with efficient verifier」を前面に出しているのは、その文脈をよく示している [27]。

8 Virgo の応用とユースケース

8.1 Virgo の位置づけ

Virgo は、transparent polynomial delegation と **GKR** 型の層還元を組み合わせた transparent proof system であり、特に **layered arithmetic circuits** に対して強い [14]。

この「layered であること」が最大の特徴である。Virgo の強さは、一般回路を平坦化するのでなく、**層構造を保ったまま claim を次層へ落とす** ことにある。

8.2 どの用途に向くか

この性質から Virgo が向くのは、

- Merkle tree のように層がはっきりした計算
- 多数の同型 subcircuit を並べるデータ並列計算
- バッチ署名検証
- layered neural network / zkML
- クロスチェーン証明やブロックヘッダ検証

である。

原論文系統および後続研究では、Merkle tree benchmark や、構造化回路に対する practical prover time が強調される。さらに zkCNN は CNN prediction correctness / accuracy の証明へ、zkBridge は cross-chain bridge へ、deVirgo は分散 proving へ発展している [14, 23, 24]。

8.3 代表ユースケース 1: zkML

zkCNN は、機械学習推論の正しさを ZK で証明する文脈で Virgo 系を用いている代表例である。ここで重要なのは、CNN の計算グラフが層構造を持ち、同型の演算が多数繰り返される点である。これは Virgo の layered / structured circuit の強みと相性がよい [23]。

したがって、**データ並列な ML inference proof** は Virgo 系の典型的な応用先の一つだと整理できる。

8.4 代表ユースケース 2: cross-chain bridge

zkBridge は、cross-chain block header relay と signature verification を trustless に証明するために、deVirgo と Groth16 wrapping を組み合わせる。論文では deVirgo が元の single-machine Virgo に比べて 100x 以上速い block header relay proof を達成し、最終的な on-chain verification cost を直接署名検証より大きく削減すると報告している [24]。

この事例は非常に示唆的である。Virgo 系は raw のまま最終出口に使われるよりも、

1. 内側で大きな structured computation を速く証明する
2. 外側を小さな公開鍵ベース proof でラップする

という形で非常に強い。これは STARK と Groth16 の関係にも似ている。

8.5 代表ユースケース 3: distributed proving

Virgo 系は分散証明と相性がよい。zkBridge の deVirgo は、同型 subcircuit を多数含む workload に対して分散化の恩恵が大きいことを示している。また Berkeley RDI の研究ページでも、distributed ZKPs により proving を multiple machines に広げる方向が強調されている [24, 25]。

このため、**「一台では重いが、複数台で並列化しやすい structured proof」**という観点では、Virgo は今回の四方式の中でも特に面白い位置にある。

8.6 Virgo の弱点と注意点

Virgo は強いが、二つ注意がある。

第一に、**一般回路をそのまま素朴に食べる系ではない**。layered / structured circuit であることが効率の源泉なので、完全に unstructured な一般回路では別の系や後継系（Virgo++ 等）が必要になる [14]。

第二に、2025 年の Revisiting Virgo は、元の zkVPD 設計・実装に limitations と exploitable vulnerabilities があったことを報告し、改善版を提示している。したがって 2026 年時点で raw 2020 Virgo をそのまま採用するより、**改良版・後継系・レビュー済み実装を選ぶ**のが妥当である [15]。

9 公開鍵ベース ZK とハッシュ関数ベース透明 ZK の包括比較

9.1 大きな傾向

本質的な違いを一言で言えば、

- **公開鍵ベース**は「proof を小さく、verification を軽く」しやすい
- **透明系**は「setup をなくし、巨大計算や監査容易性に強い」

という傾向にある。

もちろん例外はある。Halo は公開鍵ベースだが setup 不要であり、Bulletproofs は setup 不要だが verifier は重めである。そのため、正確には

$$\text{proof size / verifier cost} \quad \text{と} \quad \text{setup / transparency / } \mathbf{PQ}$$

は別軸で見るべきである [4, 6]。

9.2 比較表：公開鍵ベース vs. ハッシュベース

観点	公開鍵ベース ZK	ハッシュ関数ベース透明 ZK
setup	relation-specific, universal, updatable, あるいは不要（Halo, Bulletproofs）	原則 transparent。trusted setup 不要
主たる仮定	楕円曲線群、離散対数、ペアリング、KZG, IPA など	衝突耐性ハッシュ、Merkle 木、**RS** 符号、低次数検査、**IOP**
proof size	一般に小さい。Groth16 は特に極小	一般に大きめ。batching や wrapping で補うことが多い
verification	一般に軽い。特に on-chain 向き	方式差が大きい。raw では heavier になりやすい
prover scaling	回路系に依存。Plonkish / recursion / custom gates で改善	巨大トレースや structured computation に強い系が多い
量子耐性の見通し	一般に弱い（Shor 型脅威）	一般に良い（plausibly post-quantum）

観点	公開鍵ベース ZK	ハッシュ関数ベース透明 ZK
典型用途	L1 verifier, compact proofs, shielded tx, compact rollup receipts	zkVM, rollup backend, auditability, distributed proving, structured computation
典型ハイブリッド	大きい内部証明の外側ラッパ	内部で重い計算を証明し、最終出口は Groth16 等で圧縮

9.3 設計判断としてどう読むべきか

この表は「どちらが上か」を言うためのものではない。実際には、

- **L1 の最終レシート**には公開鍵ベース
- **内側の大規模計算証明**には透明系

という役割分担が非常に自然である。RISC Zero の Groth16Receipt と、zkBridge の deVirgo + Groth16 は、その現実例である [17, 18, 24]。

10 ハッシュ関数ベース四方式の横比較

10.1 高レベル比較

方式	主役の表現	強み	弱み	向く用途
Ligero	witness table / **RS**-encoded rows	構成が比較的素朴。transparent。中規模で使いやすい	proof 極小ではない。巨大逐次計算の主役ではない	中規模回路、Boolean predicates、compliance
zk-STARK	execution trace + **AIR** + FRI	巨大逐次計算、**zkVM**、rollup backend、**PQ** の見通し	proof が大きめ。raw on-chain verification は重い	**zkVM**、shared proving、監査用大規模計算証明
Aurora	**R1CS** + lincheck / rowcheck + FRI	既存 **R1CS** 互換。transparent	verifier が線形。raw L1 向きではない	オフチェーン **R1CS** 証明、研究プロトタイプ

方式	主役の表現	強み	弱み	向く用途
Virgo	layered circuit + **GKR** + zkVPD	structured / layered / distributed proving に強い	layering 依存。原版 Virgo には注意点	zkML、bridge、Merkle/batch sig、distributed proving

10.2 実務的な含意

この表からの実務的な含意はかなり明確である。

■STARK　最も「プロダクトの心臓部」になりやすい。とくに **zkVM** と rollup backend で強い [11, 16, 21]。

■Ligero　最も「シンプルな transparent proof」の精神を保ちやすい。中規模・Boolean 寄りの仕事に向く [8, 9]。

■Aurora　最も「**R1CS** に近い」。既存 constraint-system 文化との接続が強い [13, 27]。

■Virgo　最も「structured / distributed」に寄る。raw 単体より、後継系やハイブリッド構成を含めて理解するのが自然である [14, 24, 15]。

11 典型ユースケース別の選び方

11.1 EVM / L1 上で verifier gas を最小化したい

この用途では、まず Groth16 / PLONK / Halo2 などの公開鍵ベースを第一候補に置くのが自然である。とくに Groth16 は極めて小さい proof と軽い verification を持ち、EVM tooling も豊富である。transparent 系をそのまま L1 に投げるより、**透明系で内部証明を作って最後だけ Groth16 でラップする**方が実務的なことが多い [7, 17, 18]。

11.2 巨大なプログラム実行を証明したい

この用途では STARK 系が最有力である。理由は明確で、実行トレースと **AIR** という表現が CPU / VM execution に自然だからである。RISC Zero と Starknet / SHARP がまさにこの方向の代表例である [11, 16, 21]。

11.3 setup 不要の透明証明が欲しいが、対象は中規模回路である

この場合、Ligero 系か Aurora 系が候補になる。

- witness table 的に見やすいなら Ligero
- 既存 **R1CS** に載っているなら Aurora

という選び方が自然である。ただし verifier cost や proof size の要求が厳しいなら、公開鍵ベースや hybrid も再検討すべきである [8, 13]。

11.4 Merkle trees, batch signatures, zkML, structured pipelines

この場合は Virgo 系が有力である。特に

- 層構造がはっきりしている
- 同型 subcircuit の繰り返しが多い
- 複数マシンで prover を並列化したい

なら、Virgo / deVirgo の設計思想がよく噛み合う。zkCNN と zkBridge はこの方向を示す典型例である [23, 24]。

11.5 量子耐性や長期監査を重視する

この場合は transparent hash-based 系に大きな意味がある。とりわけ STARK は、「post-quantum secure computational integrity」を出発点から掲げているため、長期保存・規制説明・将来監査の文脈で語りやすい [11]。

11.6 完全に one-size-fits-all を求めない

現代の実システムでは、単一方式ですべての要求を満たすことはむしろ稀である。proof backend と final receipt を分離し、

- 内側：transparent / scalable / distributed
- 外側：tiny / on-chain friendly

とするハイブリッドが現実的である [17, 24]。

12 ハイブリッド構成が重要である理由

12.1 RISC Zero: STARK の上に Groth16

RISC Zero の proving stack は、内部に RISC-V Circuit と Recursion Circuit を持ち、最終的に Groth16 Circuit で tiny receipt に圧縮する。これは

- 大きな実行証明は **STARK** 系で
- 最終配布・検証は Groth16 で

という分業の見本である [17, 18]。

12.2 zkBridge: deVirgo の上に Groth16

zkBridge でも、block header relay と署名検証の重い部分は deVirgo で捌き、最終的な on-chain verification は Groth16 の compactness を使う。この構成は、**透明・分散・大規模 proving** と**小さな最終証明** を両立させる代表例である [24]。

12.3 理論的含意

この二例から得られる重要な教訓は、

> ハッシュ関数ベース透明系と公開鍵ベース系は、競合関係であると同時に、非常に強い補完関係にある。

ということである。

したがって設計上の問いは「どちらが勝つか」ではなく、

> **どの層で何を証明し、どの層で何を小さくするか**

である。

13 補足的な考察

13.1 透明であることの非技術的価値

transparent であることは、純粋な暗号学的性質以上の意味を持つ。ceremony の不備や毒性廃棄（toxic waste）の懸念は、技術者だけでなく監査人、規制担当、経営層の説明対象になる。そのため、setup が不要であることは

- 組織内承認
- 対外監査
- ガバナンス説明

においても価値がある。特に複数組織が関わる bridge や証明インフラでは、この利点は小さくない [11, 8, 13, 14]。

13.2 「透明系 = 何でも PQ」ではない

本稿で「plausibly post-quantum」と書いたのは意図的である。Fiat–Shamir の扱い、ハッシュの安全性、細部の argument、実装の side channel やソフトウェアバグまで含めると、「量子計算でも完全に安心」と強く言い切るのは危険である。とはいえ、離散対数や pairings を主要仮定にする系

より、長期的な説明可能性の面で transparent hash-based 系が有利なのは確かである [11, 15]。

13.3 Virgo は原版より系譜で捉える方が良い

2025 年の Revisiting Virgo により、Virgo は「古典として読む」だけでなく、「どのように修正・継承されたか」を含めて理解する必要があることが明確になった。今日の実務的な態度としては、

- Virgo の発想（layered, structured, distributed）を評価する
- ただし原版実装をそのまま信用しない
- 後続の修正版・後継系・レビュー済み実装を使う

という整理が妥当である [15]。

13.4 Aurora と Ligero の現在位置

Aurora と Ligero は、いま最前線の「生のプロダクト名」として出てくることは STARK ほど多くないかもしれない。しかしその影響は大きい。Aurora は transparent **R1CS** proof の重要な祖先であり、Ligero は transparent sublinear proof の重要祖先である。したがってこの二方式は、**現代 transparent proof の発想の出発点**として非常に重要である [8, 13, 27, 10]。

14 実務的な結論

最後に、設計判断としてかなり率直にまとめる。

1. L1 / EVM で最終検証コストが最優先

公開鍵ベース、特に Groth16 / PLONK / Halo2 系をまず検討する。巨大な内部証明が必要なら transparent 系を使ってもよいが、最終出口は Groth16 などでラップするのが自然である [7, 17, 24]。

2. 巨大な逐次計算・zkVM・rollup backend

STARK を第一候補に置く。Starknet と RISC Zero はその現実例である [16, 21]。

3. setup 不要の透明 proof を中規模回路に使いたい

Ligero 系または Aurora 系を検討する。Boolean 寄りなら Ligero 系、既存 **R1CS** / TinyRAM 資産があるなら Aurora 系が自然である [8, 13, 9]。

4. structured / layered / data-parallel / distributed

Virgo 系が魅力的である。ただし後継系・修正版を前提に設計する [14, 24, 15]。

5. 長期監査・量子耐性・ガバナンス説明

transparent hash-based 系の価値が大きい。特に STARK はこの文脈で最も説得力が強い [11]。

15 結論

本稿の主張は次のように要約できる。

1. **公開鍵ベース ZK** は、小さな proof と軽い verification を必要とする用途、とりわけ on-chain verification に強い。Groth16, PLONK, Marlin, Halo, Bulletproofs は、その内部でも setup・recursion・verification cost の違いで棲み分ける [1, 2, 3, 4, 5]。
2. **ハッシュ関数ベース透明 ZK** は、setup 不要、監査容易性、**PQ** の見通し、そして大規模計算や structured computation への適性で強い。その中でも
   - STARK は **zkVM** / rollup / execution trace
   - Ligero は中規模・Boolean・シンプルな transparent proof
   - Aurora は transparent **R1CS** / TinyRAM
   - Virgo は layered / structured / distributed proving

   にそれぞれ向く [8, 11, 13, 14, 24, 23]。
3. 現代の実務では、**透明系と公開鍵ベース系を組み合わせるハイブリッド構成**が極めて重要である。STARK や deVirgo が大きな内部計算を処理し、Groth16 が最終的な tiny receipt を作る、という分業はすでに実用例がある [17, 18, 24]。

したがって設計者が本当に問うべきなのは、「どの方式が一番良いか」ではなく、

どの層で何を証明し、どの層で何を小さくし、どの信頼仮定をどこに置くか

である。この問いに対しては、単独方式よりも、**内部証明と最終レシートを分離した多層設計**がしばしば最も現実的である。

参考文献

[1] Jens Groth. On the Size of Pairing-Based Non-interactive Arguments. In *EUROCRYPT 2016*, LNCS 9666, pp. 305–326, 2016. DOI: https://doi.org/10.1007/978-3-662-49896-5_11.

[2] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge. IACR ePrint 2019/953. https://eprint.iacr.org/2019/953.

[3] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas Ward. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. IACR ePrint

2019/1047. https://eprint.iacr.org/2019/1047.

[4] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo: Recursive Proof Composition without a Trusted Setup. IACR ePrint 2019/1021. https://eprint.iacr.org/2019/1021.

[5] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Gregory Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. IEEE Symposium on Security and Privacy, 2018. https://eprint.iacr.org/2017/1066.

[6] Stanford Applied Crypto Group. Bulletproofs. https://crypto.stanford.edu/bulletproofs/.

[7] iden3. snarkjs-generate-solidity README. https://github.com/iden3/snarkjs-generate-solidity/blob/main/README.md.

[8] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight Sublinear Arguments Without a Trusted Setup. ACM CCS 2017. https://acmccs.github.io/papers/p2087-amesA.pdf.

[9] Yaron Gvili, Sarah Scheffler, and Mayank Varia. Improved Sublinear Zero Knowledge Proofs for Boolean Circuits. IACR ePrint 2021/121. https://eprint.iacr.org/2021/121.

[10] Rishabh Bhadauria, Pratyush Mishra, and Divya Ravi. Ligero++: A New Optimized Sublinear IOP. ACM CCS 2020. https://dl.acm.org/doi/pdf/10.1145/3372297.3417893.

[11] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, Transparent, and Post-Quantum Secure Computational Integrity. IACR ePrint 2018/046. https://eprint.iacr.org/2018/046.

[12] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon Interactive Oracle Proofs of Proximity. ICALP 2018. https://drops.dagstuhl.de/storage/00lipics/lipics-vol107-icalp2018/LIPIcs.ICALP.2018.14/LIPIcs.ICALP.2018.14.pdf.

[13] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent Succinct Arguments for R1CS. IACR ePrint 2018/828. https://eprint.iacr.org/2018/828.

[14] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, Boyuan Feng, and Dawn Song. Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof Systems. IACR ePrint 2019/1482. https://eprint.iacr.org/2019/1482.

[15] Changchang Ding and Yan Huang. Revisiting Virgo: A Study of Vulnerabilities, Limitations, and Optimizations. *Cybersecurity*, 2025. https://link.springer.com/article/10.1186/s42400-025-00450-w.

[16] Starknet Documentation. SHARP. https://docs.starknet.io/learn/protocol/sharp.

[17] RISC Zero Developer Docs. Proof System Overview. https://dev.risczero.com/proof-system/.

[18] RISC Zero Developer Docs. Recursive Proving. https://dev.risczero.com/api/

recursion.

[19] RISC Zero Developer Docs. The RISC Zero STARK Protocol / Proof System Sequence Diagram. https://dev.risczero.com/proof-system/proof-system-sequence-diagram.

[20] RISC Zero Developer Docs. About PLONK and PLOOKUP. https://dev.risczero.com/reference-docs/about-plonk.

[21] RISC Zero. Scalable, Transparent Arguments of RISC-V Integrity. https://dev.risczero.com/proof-system-in-detail.pdf.

[22] Zcash Documentation. Network Upgrade Guide (NU5 / Halo). https://zcash.readthedocs.io/en/latest/rtd_pages/nu_dev_guide.html.

[23] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. ACM CCS 2021. https://eprint.iacr.org/2021/673.

[24] Tiancheng Xie, Chao Zhang, Jiaheng Zhang, Yupeng Zhang, Dawn Song, and others. zkBridge: Trustless Cross-chain Bridges Made Practical. arXiv:2210.00264, 2022. https://arxiv.org/abs/2210.00264.

[25] Berkeley RDI. Zero-Knowledge Proof Research. https://rdi.berkeley.edu/zkp/.

[26] Anne Canteaut et al. Banquet: Short and Efficient Quantum-Safe Proofs for AES and SHA-256. IACR ePrint 2021/068. https://eprint.iacr.org/2021/068.

[27] Shengyu Fu, Xiong Fan, et al. Polaris: Transparent Succinct Zero-Knowledge Arguments for R1CS with Efficient Verifier. PoPETs 2022. https://petsymposium.org/popets/2022/popets-2022-0027.pdf.